Arquitectura de Computadores

Práctica 1

Jerarquía de Memoria y Comportamiento de Caché

Estudio del efecto de la localidad de los accesos a memoria en las prestaciones de programas en microprocesadores

---

Plataforma:	FinisTerrae III (CESGA)
Procesador:	Intel Xeon Platinum 8352Y (Ice Lake-SP)
Compilador:	`gcc -O0`
Strides D:	1, 8, 16, 64, 512 (5 valores, potencias de 2)
Tamaños L:	11 valores, de 384 a 3 145 728 líneas
Mediciones:	$3 \times 5 \times 11 \times 10 = 1\,650$ totales
Métrica:	Media geométrica de los 3 mejores de 10 repeticiones

March 4, 2026

Contents

1 Introducción y objetivos **3**

2 Arquitectura del sistema **3**
2.1 Jerarquía de memoria . . . 3
2.2 Prefetcher hardware (microarquitectura Sunny Cove) . . . 3

3 Metodología **4**
3.1 Parámetros del experimento . . . 4
3.2 Cálculo de R . . . 4
3.3 Configuración Slurm: nodo dedicado con `-c 64` . . . 4
3.4 Tratamiento estadístico . . . 5

4 Diseño Experimental, Códigos y Saltos de Caché **5**
4.1 Selección de Valores de Experimentación . . . 5
4.2 Desarrollo de los Códigos en C . . . 5
4.3 Análisis de la cantidad de accesos y saltos de caché . . . 5

5 Implementación **6**
5.1 `acp1.c` — experimento base . . . 6
5.2 `acp1_directo.c` y `acp1_int.c` . . . 6

6 Resultados experimentales **7**
6.1 Double + acceso indirecto (`acp1.c`) . . . 7
6.2 Int + acceso indirecto (`acp1_int.c`) . . . 7
6.3 Double + acceso directo (`acp1_directo.c`) . . . 8

7 Gráficas de resultados **8**
7.1 Experimento base con 5 strides . . . 8
7.2 Comparativa global de los tres experimentos . . . 9
7.3 Comparativa por stride: D=1 y D=16 . . . 10

8 Análisis por zonas de la jerarquía **11**
8.1 Zona L1 — $L \leq 768$ líneas (footprint $\leq$ 48 KB) . . . 11
8.2 Zona L2 — $768 < L \leq 20\,480$ líneas (footprint 72 KB–960 KB) . . . 11
8.3 Zona L3 — $20\,480 < L \leq 786\,432$ líneas (footprint 2.5 MB–48 MB) . . . 11
8.4 Zona RAM — $L > 786\,432$ líneas (footprint > 48 MB) . . . 11

9 La escalera completa de la jerarquía **12**

10 Análisis comparativo de variantes **12**
10.1 Double vs. Int . . . 12
10.2 Directo vs. Indirecto . . . 13

11 Latencias identificadas experimentalmente **13**

12 Conclusiones **13**

A Configuración Slurm **14**

1 Introducción y objetivos

Esta práctica caracteriza experimentalmente la latencia de cada nivel de la jerarquía de memoria del procesador Intel Xeon Platinum 8352Y mediante una operación de reducción de suma sobre un vector con patrón de acceso controlado. Se varían dos parámetros:

- **D (stride):** separación en elementos entre posiciones accedidas. Controla la localidad espacial y la eficacia del hardware prefetcher.
- **L (líneas):** número de líneas de caché distintas referenciadas. Determina en qué nivel de la jerarquía residen los datos durante el experimento.

El experimento base accede de forma **indirecta** mediante el vector de índices `ind[]`. Se realizan dos experimentos adicionales: con tipo `int` (4 bytes en lugar de 8) y con acceso **directo** eliminando `ind[]`.

2 Arquitectura del sistema

2.1 Jerarquía de memoria

Table 1: Jerarquía de caché del Intel Xeon Platinum 8352Y (Ice Lake-SP)

Nivel	Capacidad	Asoc.	S (líneas)	Latencia aprox.
L1d	48 KB	12-way	$S_1 = 768$	∼5 ciclos
L2	1.25 MB	20-way	$S_2 = 20\,480$	∼14 ciclos
L3	48 MB	12-way	$S_3 = 786\,432$	∼40–50 ciclos
RAM	256 GB DDR4	—	—	∼150–200 ciclos

Tamaño de línea de caché: **CLS = 64 bytes**. Los experimentos se ejecutan en **un único core** con el nodo completo reservado (`-c 64`).

2.2 Prefetcher hardware (microarquitectura Sunny Cove)

El procesador implementa cuatro prefetchers hardware independientes:

DCU Streamer: Precarga la siguiente línea de caché ante accesos ascendentes. Se activa con cualquier patrón secuencial.

DCU IP Stride Prefetcher: Rastrea instrucciones de carga individuales. Cuando detecta un stride regular, emite precargas anticipadas hasta 2 KB de distancia.

L2 Streamer: Monitoriza peticiones desde L1 y corre hasta **20 líneas por delante**. Gestiona hasta 32 streams. No cruza límites de página de 4 KB.

L2 Spatial (Adjacent Cache Line): Completa cada línea cargada en L2 con su línea adyacente (bloques de 128 bytes). Su actuación independiente del patrón provoca *prefetch pollution* con $D = 16$.

3 Metodología

3.1 Parámetros del experimento

Strides D

Se usan **5 valores** potencia de 2: $D \in \{1, 8, 16, 64, 512\}$. Estos valores fueron seleccionados para cubrir representativamente los distintos regímenes de localidad espacial:

- $D = 1$: máxima localidad espacial, 8 doubles por línea de caché, prefetcher óptimo.
- $D = 8$: exactamente un acceso por línea de caché (stride = 64 bytes = CLS).
- $D = 16$: stride de 128 bytes, comienza a saltar líneas; el L2 Spatial Prefetcher genera *prefetch pollution*.
- $D = 64$: stride de 512 bytes, localidad espacial nula, prefetcher ineficaz.
- $D = 512$: stride de 4096 bytes, caso extremo; el stride iguala el tamaño de página, lo que puede provocar conflictos de *page aliasing*.

Valores de L

Table 2: Valores de L y nivel de jerarquía correspondiente

L (líneas)	Fracción	Footprint	Nivel
384	$0.5 \times S_1$	24 KB	L1 (holgado)
1 152	$1.5 \times S_1$	72 KB	L2 (desborda L1)
10 240	$0.5 \times S_2$	640 KB	L2 (holgado)
15 360	$0.75 \times S_2$	960 KB	L2 (ajustado)
40 960	$2 \times S_2$	2.5 MB	L3 (desborda L2)
81 920	$4 \times S_2$	5 MB	L3
163 840	$8 \times S_2$	10 MB	L3
524 288	$0.67 \times S_3$	32 MB	L3 (llenando)
786 432	$1 \times S_3$	48 MB	Límite exacto L3
1 572 864	$2 \times S_3$	96 MB	**RAM real**
3 145 728	$4 \times S_3$	192 MB	**RAM profunda**

3.2 Cálculo de R

$$R = \frac{L \times \text{CLS}}{D \times \text{sizeof(tipo)}}, \qquad N = (R - 1) \cdot D + 1$$

3.3 Configuración Slurm: nodo dedicado con `-c 64`

Los tres scripts de Slurm se configuran con `#SBATCH -c 64`, reservando los **64 cores lógicos del nodo completo** aunque solo se utilice uno para la medición. Esta decisión es crucial por cuatro razones:

1. Garantiza que ningún otro proceso del sistema de colas comparte el nodo, eliminando contaminación por tráfico de caché externo.
2. Asegura que L1 y L2 (privadas por core) son exclusivas del proceso de medición.
3. Evita que el planificador del SO migre el proceso a otro core con cachés en estado diferente.
4. Reduce la variabilidad en la zona L3 (compartida entre los 36 cores físicos del socket), donde otros procesos provocarían expulsiones de líneas ajenas.

3.4 Tratamiento estadístico

Para cada combinación (D, L): 10 repeticiones externas, media geométrica de los 3 mejores valores de ciclos/acceso.

4 Diseño Experimental, Códigos y Saltos de Caché

4.1 Selección de Valores de Experimentación

Los valores de experimentación fueron seleccionados basándose en la arquitectura del procesador. Se identificó el número de líneas límite de cada nivel dividiendo su capacidad por el tamaño de línea de caché (64 bytes): $S_1 = 768$ (L1d), $S_2 = 20\,480$ (L2) y $S_3 = 786\,432$ (L3). A partir de estos topes, los valores de L se escogieron usando fracciones o múltiplos representativos (por ejemplo, $0.5 \times S_1$, $1.5 \times S_1$, $4.0 \times S_3$), lo que garantiza probar escenarios donde los datos residen cómodamente dentro de un nivel, desbordan al siguiente, o alcanzan la RAM profunda (hasta 192 MB).

4.2 Desarrollo de los Códigos en C

Se implementaron tres variantes en lenguaje C para estudiar las diferencias arquitectónicas:

- `acp1.c` **(Base doble indirecto):** Evalúa un array tipo `double` de 8 bytes con un patrón de acceso indirecto (`A[ind[i]]`), requiriendo dos lecturas de memoria por iteración.
- `acp1_int.c` **(Entero indirecto):** Usa tipo `int` de 4 bytes. Caben el doble de elementos por línea de caché, por lo que R se duplica respecto al caso double para el mismo L y D.
- `acp1_directo.c` **(Doble directo):** Elimina `ind[]` accediendo directamente como `A[i * D]`, liberando capacidad de caché y mejorando la previsibilidad del patrón de acceso.

4.3 Análisis de la cantidad de accesos y saltos de caché

El salto de caché depende del tamaño de la línea (**64 bytes**) y del tipo de dato.

- **Para tipo `double` (8 bytes):** Caben exactamente 8 elementos por línea. Con $D = 1$ se realizan 8 accesos antes de forzar un salto; con $D = 8$ cada acceso toca una línea distinta (stride = 64 bytes = CLS).
- **Para tipo `int` (4 bytes):** Caben 16 elementos por línea. Con $D = 1$ se realizan 16 accesos antes de saltar; con $D = 16$ se fuerza exactamente un acceso por línea ($16 \times 4 = 64$ bytes).

5 Implementación

5.1 `acp1.c` — experimento base

```c
// R = lineas * bytes/linea / (stride * bytes/elem)
long long R = (long long)L * CLS / (D * sizeof(double));
long long N = (R - 1) * D + 1;
// Reserva alineada: A[0] coincide con inicio de linea de cache
double *A   = (double*)aligned_alloc(CLS, N * sizeof(double));
int    *ind = (int*)malloc(R * sizeof(int));
double  S[REPS]; // Resultados de cada repeticion

// Inicializacion en [1,2) con signo aleatorio (calentamiento + anti-overflow)
for (long long i = 0; i < R; i++) ind[i] = (int)(i * D);
for (long long i = 0; i < R; i++) {
    double val = 1.0 + (double)rand() / RAND_MAX;
    if (rand() % 2) val = -val;
    A[ind[i]] = val;
}

// Medicion de ciclos
start_counter();
for (int k = 0; k < REPS; k++) {
    double suma = 0.0;
    for (long long i = 0; i < R; i++)
        suma += A[ind[i]]; // Acceso indirecto
    S[k] = suma;
}
double ciclos_totales = get_counter();
double ciclos_por_acceso = ciclos_totales / ((double)R * REPS);

// Imprimir S[] (requerido por el enunciado)
for (int k = 0; k < REPS; k++) printf(" %.6f", S[k]);
```

Listing 1: Cálculo de R y bucle de medición (acp1.c)

5.2 `acp1_directo.c` y `acp1_int.c`

```c
// --- acp1_directo.c: acceso directo sin ind[] ---
for (long long i = 0; i < R; i++)
    suma += A[i * D]; // Un unico acceso a memoria por iteracion

// --- acp1_int.c: tipo int (4 bytes, R = el doble que double) ---
long long R = (long long)L * CLS / (D * sizeof(int)); // R*2
```

```
int *A = (int*)aligned_alloc(CLS, N * sizeof(int));
int suma = 0;
for (long long i = 0; i < R; i++)
    suma += A[ind[i]]; // ADD entero: menor latencia que FADD
```

Listing 2: Diferencias clave de las variantes adicionales

6 Resultados experimentales

Código de colores en las tablas: verde < 8 ciclos / naranja 8–12 ciclos / rojo > 12 ciclos.

6.1 Double + acceso indirecto (`acp1.c`)

Table 3: Ciclos/acceso — Double indirecto. 5 strides × 11 valores de L.

L	Nivel	D=1	D=8	D=16	D=64	D=512
384	L1	7.08	7.15	7.07	7.12	8.88
1 152	L2	7.09	7.17	7.19	7.16	7.64
10 240	L2	7.10	7.17	7.18	7.23	7.46
15 360	L2	7.11	7.21	7.25	7.30	7.49
40 960	L3	7.12	7.35	7.68	7.74	7.79
81 920	L3	7.12	7.29	7.58	7.61	7.63
163 840	L3	7.12	7.27	7.85	7.60	7.56
524 288	L3	7.18	8.82	12.86	13.74	13.23
786 432	L3 lím.	7.19	9.38	14.71	14.78	15.65
1 572 864	**RAM**	7.19	9.82	16.22	16.59	17.20
3 145 728	**RAM**	7.20	9.81	17.09	17.77	16.93

6.2 Int + acceso indirecto (`acp1_int.c`)

Table 4: Ciclos/acceso — Int indirecto.

L	Nivel	D=1	D=8	D=16	D=64	D=512
384	L1	6.76	6.79	6.85	6.80	7.55
1 152	L2	6.76	6.82	7.17	7.24	7.39
10 240	L2	6.78	6.83	7.16	7.24	7.26
15 360	L2	6.80	6.84	7.23	7.25	7.27
40 960	L3	6.80	6.89	7.31	7.77	7.53
81 920	L3	6.79	6.88	7.26	7.62	7.52
163 840	L3	6.79	6.89	7.31	7.86	7.59
524 288	L3	6.81	7.47	8.70	14.35	14.10
786 432	L3 lím.	6.81	7.55	9.31	15.96	14.86
1 572 864	**RAM**	6.81	7.61	9.74	17.59	15.74
3 145 728	**RAM**	6.81	7.61	9.79	18.51	16.33

6.3 Double + acceso directo (`acp1_directo.c`)

Table 5: Ciclos/acceso — Double directo.

L	Nivel	D=1	D=8	D=16	D=64	D=512
384	L1	7.06	6.97	6.81	6.28	8.03
1 152	L2	7.08	7.09	7.07	6.83	7.01
10 240	L2	7.10	7.14	7.16	7.10	7.00
15 360	L2	7.11	7.26	7.20	7.22	7.31
40 960	L3	7.11	7.37	7.46	7.48	7.33
81 920	L3	7.12	7.30	7.48	7.31	7.46
163 840	L3	7.11	7.29	7.50	7.33	7.33
524 288	L3	7.15	8.38	11.98	12.36	12.21
786 432	L3 lím.	7.16	8.81	13.43	13.66	14.01
1 572 864	**RAM**	7.17	9.21	15.05	15.45	15.78
3 145 728	**RAM**	7.17	9.18	15.86	16.21	16.08

7 Gráficas de resultados

7.1 Experimento base con 5 strides

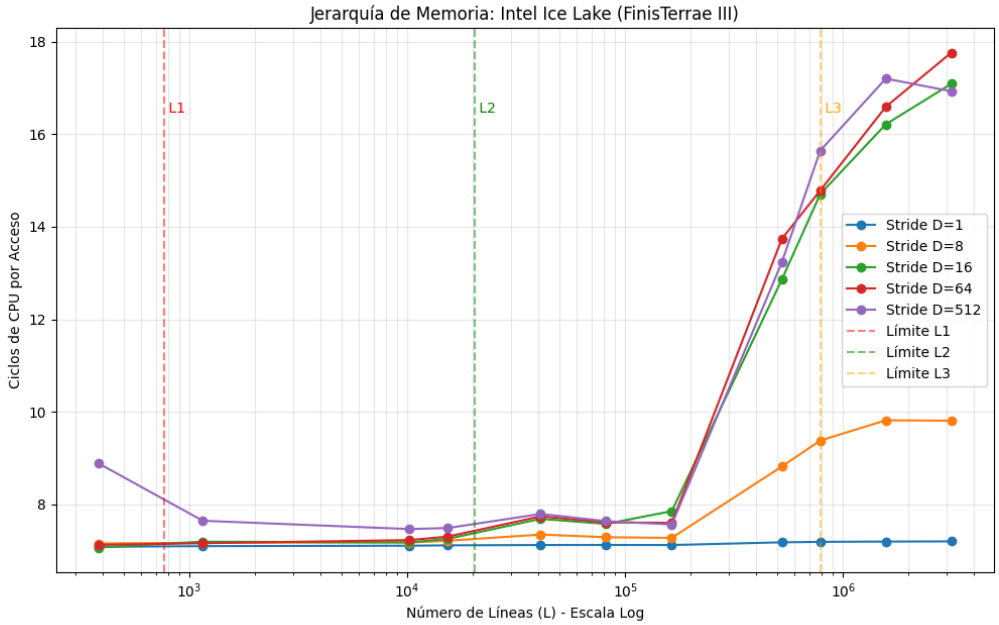


Figure 1: Ciclos/acceso vs. líneas L (escala log) para el experimento base (double + indirecto, 5 strides). Las líneas verticales marcan los límites de L1 ($S_1 = 768$, rojo), L2 ($S_2 = 20\,480$, verde) y L3 ($S_3 = 786\,432$, naranja). Se aprecian claramente tres regiones: zona plana L1/L2 con prefetcher eficaz, escalada pronunciada a partir de L3 alta, y estabilización en RAM.

7.2 Comparativa global de los tres experimentos

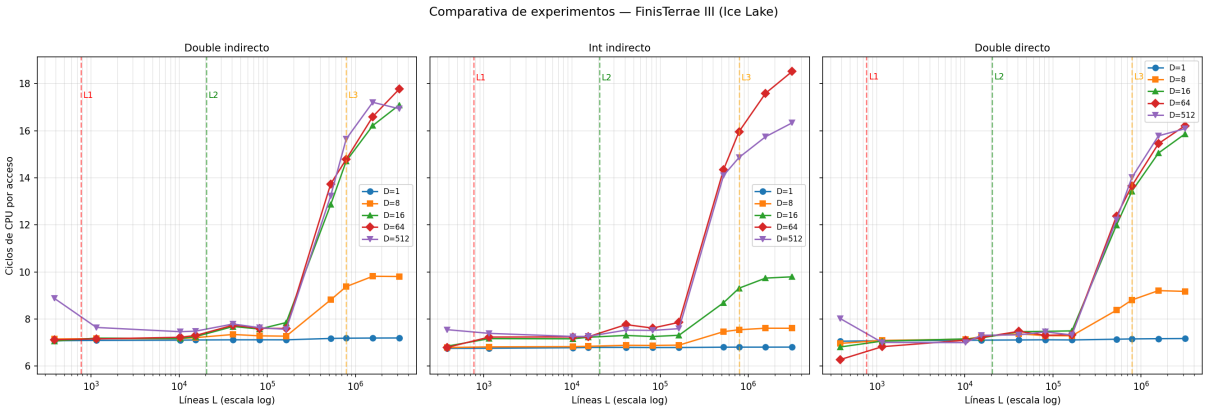
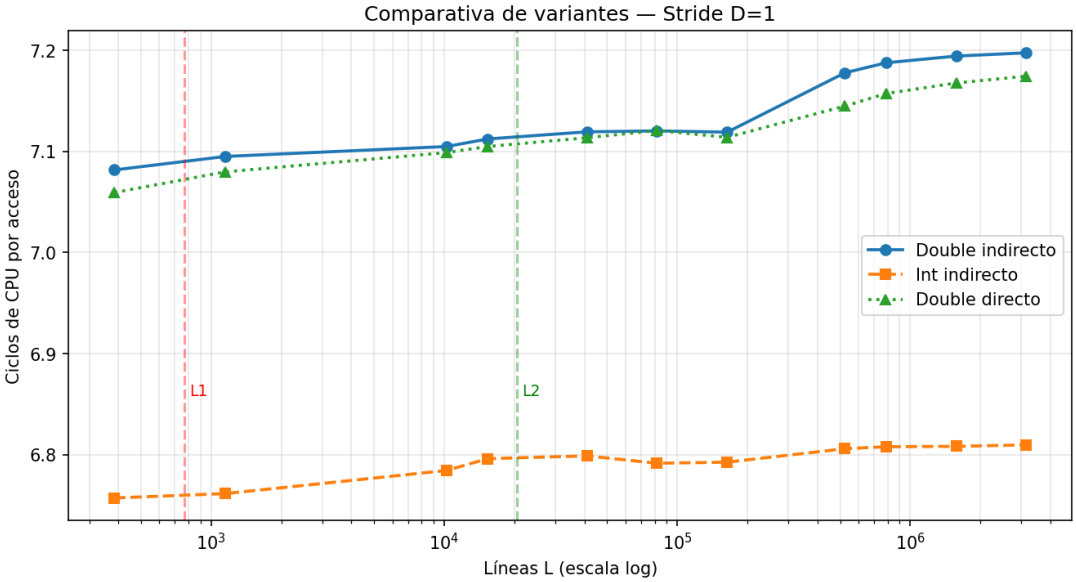
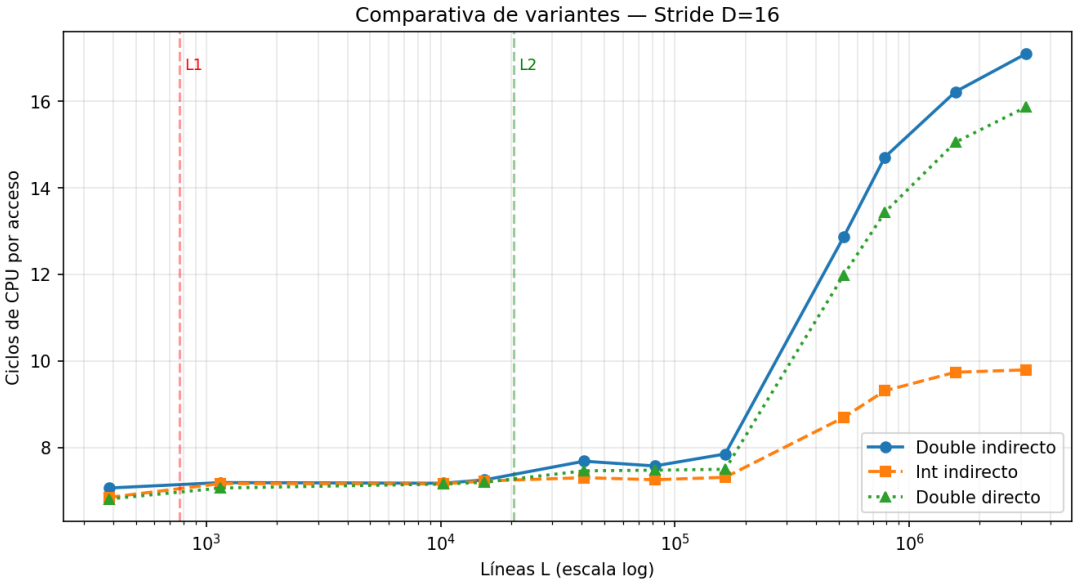


Figure 2: Comparativa de los tres experimentos en tres paneles paralelos, cada uno con los 5 strides. Las líneas verticales marcan S_1 (rojo), S_2 (verde) y S_3 (naranja). A partir de $L = 524\,288$ los datos desbordan L3 y la latencia escala abruptamente para $D \geq 16$.

7.3 Comparativa por stride: D=1 y D=16



(a) Stride $D = 1$: las tres variantes permanecen completamente planas en toda la jerarquía, incluyendo RAM con 192 MB de footprint. El hardware prefetcher oculta por completo todas las latencias. La variante int es ∼0.3 ciclos más rápida por la menor latencia de la suma entera.



(b) Stride $D = 16$: el double indirecto alcanza ∼17.1 ciclos en RAM. El int indirecto se mantiene por debajo de 10 ciclos gracias a la mayor densidad de datos útiles por línea de caché. El double directo queda entre ambas por la menor presión de caché al eliminar `ind[]`.

Figure 3: Comparativa de las tres variantes para los strides más representativos.

8 Análisis por zonas de la jerarquía

8.1 Zona L1 — $L \leq 768$ líneas (footprint $\leq$ 48 KB)

Con $L = 384$ (24 KB de footprint), los datos caben completamente en la L1d de 48 KB. Las curvas convergen en el rango 6.3–7.7 ciclos para todos los strides y variantes. El único comportamiento anómalo es $D = 512$ con $L = 384$: con tan pocos elementos ($R = 6$ para double), el overhead del bucle con `-O0` (control de iteración, cálculo de $i \times D$ en cada paso) domina sobre el coste del acceso real, inflando el valor a 8.88 ciclos. Esto no refleja la latencia de L1 sino el coste computacional del bucle.

La ventaja de `int` ($\sim$0.3 ciclos menos que double) se debe a que la instrucción de suma entera `ADD` tiene menor latencia que `FADD` de punto flotante. La ventaja del acceso directo con $D = 64$ (6.28 vs. 7.12 ciclos, mejora del 12%) refleja la eliminación de la lectura de `ind[]`, que con $D = 64$ supone el acceso a una línea de caché adicional por iteración.

8.2 Zona L2 — $768 < L \leq 20\,480$ líneas (footprint 72 KB–960 KB)

A pesar de desbordar L1 (latencia teórica de L2 $\sim$14 ciclos), los valores medidos se mantienen por debajo de 7.5 ciclos en todos los casos. El **DCU IP Stride Prefetcher** detecta el stride constante y emite precargas desde L2 hacia L1 antes de que sean necesarias, ocultando virtualmente toda la latencia de L2. La penalización al cruzar de L1 a L2 es inferior a 0.2 ciclos, confirmando la eficacia del prefetcher incluso para strides de hasta $D = 512$.

8.3 Zona L3 — $20\,480 < L \leq 786\,432$ líneas (footprint 2.5 MB–48 MB)

Se distinguen dos subzonas. En la **L3 baja** ($L = 40\,960$–$163\,840$, footprint 2.5–10 MB), todos los strides se mantienen por debajo de 8 ciclos: el L2 Streamer sigue siendo eficaz. En la **L3 alta** ($L = 524\,288$–$786\,432$, 32–48 MB) se produce la transición crítica:

$D = 1$: Permanece plano en $\sim$7.2 ciclos. El L2 Streamer (20 líneas por delante) oculta la latencia de L3 con la misma eficacia que la de L2.

$D = 8$: Sube progresivamente hasta $\sim$9.4 ciclos. El prefetcher sigue activo pero con menor eficacia al crecer la distancia entre accesos.

$D = 16$: Peor caso ($\sim$14.7 ciclos en el límite L3). El L2 Spatial Prefetcher genera *prefetch pollution*: por cada línea útil de 64 bytes trae otra adyacente innecesaria, duplicando el tráfico L2–L3.

$D = 64$ y $D = 512$: Latencias similares (13–16 ciclos). El stride supera el umbral del L2 Streamer pero, al no haber pollution, no empeoran respecto a $D = 16$.

8.4 Zona RAM — $L > 786\,432$ líneas (footprint $>$ 48 MB)

La transición L3→RAM es el hallazgo más relevante. Entre $L = 786\,432$ y $L = 1\,572\,864$ se produce un **cambio de pendiente** apreciable para $D \geq 8$. Los valores se estabilizan entre $L = 1\,572\,864$ y $L = 3\,145\,728$ (diferencia inferior al 2%), confirmando que se ha alcanzado la latencia de RAM pura.

Table 6: Ciclos/acceso en RAM profunda ($L = 3\,145\,728$, footprint 192 MB)

Variante	**D=1**	**D=8**	**D=16**	**D=64**	**D=512**
Double indirecto	7.20	9.81	17.09	17.77	16.93
Int indirecto	6.81	7.61	9.79	18.51	16.33
Double directo	7.17	9.18	15.86	16.21	16.08
Mejor	Int	Dir.	Int	Dir.	Dir.

Observación clave: $D = 1$ es completamente inmune a la jerarquía. Con 192 MB en RAM el coste es ∼7.2 ciclos, idéntico a L1. Los cuatro prefetchers actúan coordinadamente cubriendo todos los niveles.

9 La escalera completa de la jerarquía

Con 11 valores de L se puede trazar la escalera completa. Para D=64 (double indirecto):

Table 7: Escalera de latencias para D=64, double indirecto

Zona	**L**	**Footprint**	**Ciclos/acceso**
L1	384	24 KB	7.12
L2	15 360	960 KB	7.30 (+0.2)
L3 entrada	40 960	2.5 MB	7.74 (+0.4)
L3 profunda	786 432	48 MB	14.78 (+7.0)
RAM	3 145 728	192 MB	17.77 (+3.0)

La diferencia L1→RAM es de $\Delta = 10.65$ ciclos, frente a los ∼150 teóricos de RAM pura. El prefetcher atenúa la penalización en un factor de $\approx$ **14**×.

10 Análisis comparativo de variantes

10.1 Double vs. Int

En la zona L1/L2, `int` es ∼0.3 ciclos más rápido por la menor latencia de `ADD`. En L3/RAM con strides medios ($D = 16$), la ventaja es mucho mayor: en RAM profunda, `int` obtiene 9.79 ciclos frente a 17.09 del double (mejora del **43%**). Esto se debe a que con el doble de R la relación entre elementos útiles accedidos y líneas de `ind[]` leídas mejora, reduciendo la presión relativa del vector de índices.

Sin embargo, para strides grandes en RAM ($D = 64$), `int` es el *peor* con 18.51 ciclos, superando al double indirecto (17.77). La razón: con $R_{\text{int}} = 2 \times R_{\text{double}}$, el vector `ind[]` tiene el doble de entradas y genera accesos adicionales masivos a RAM para leer los propios índices.

10.2 Directo vs. Indirecto

La eliminación de `ind[]` beneficia especialmente a strides grandes en L1 ($D = 64$: 6.28 vs. 7.12 ciclos, mejora del 12%) y a la zona RAM con $D = 8$ (9.18 vs. 9.81 ciclos, mejora del 6%). En RAM con $D \geq 64$ la ventaja se reduce porque el cuello de botella es la latencia de RAM en sí misma y no la contención entre los dos streams de acceso (datos e índices).

11 Latencias identificadas experimentalmente

Table 8: Latencias teóricas vs. observadas y factor de mejora del prefetcher

Nivel	Lat. teórica	Ciclos observados	Factor mejora
L1d	∼5 ciclos	∼7 ciclos	— (overhead `-O0`)
L2	∼14 ciclos	∼7 ciclos	2× (latencia oculta)
L3	∼45 ciclos	7–15 ciclos	3×–6×
RAM	∼150 ciclos	7–18 ciclos	8×–21×

Los valores en L1 (∼7 ciclos) superan la latencia teórica de 5 ciclos por el overhead del bucle sin optimización (`-O0`): el compilador no elimina el cálculo de `i*D` ni el acceso indirecto a `ind[]`.

12 Conclusiones

1. **La experimentación cubre toda la jerarquía.** Con 11 valores de L desde 24 KB hasta 192 MB se caracterizan experimentalmente los cuatro niveles. Los límites $S_1 = 768$, $S_2 = 20\,480$ y $S_3 = 786\,432$ líneas se validan por los cambios de pendiente en las curvas.
2. **`-c 64` es esencial para la fiabilidad.** Reservar el nodo completo elimina la contaminación por otros procesos y garantiza la exclusividad de L1 y L2, reduciendo la variabilidad de las mediciones.
3. **D=1 es inmune a la jerarquía.** Los cuatro prefetchers actúan coordinadamente ocultando las latencias de L2, L3 y RAM. Con 192 MB en RAM el coste es ∼7.2 ciclos, idéntico a L1.
4. **La escalera de latencias aparece muy atenuada.** La diferencia L1→RAM para $D = 64$ es de 10.65 ciclos en lugar de los ∼150 teóricos (14× de mejora por el prefetcher).
5. **La transición L3→RAM es claramente observable** entre $L = 786\,432$ y $L = 1\,572\,864$ para $D \geq 8$, validando experimentalmente $S_3 = 786\,432$ líneas.

6. **D=16 es el peor stride** en double indirecto (17.09 ciclos en RAM profunda) por el fenómeno de *prefetch pollution* del L2 Spatial Prefetcher, que trae una línea adyacente innecesaria por cada línea útil.
7. **D=512 no empeora significativamente respecto a D=64.** A partir de cierto stride el prefetcher es igualmente ineficaz y la latencia se estabiliza, sin degradación monótona adicional.
8. **El acceso directo supera al indirecto** en zona L1 con strides grandes (12% de mejora con $D = 64$) y en RAM con strides medios (6% con $D = 8$).
9. **El tipo `int` supera al `double`** para strides $D = 8$ y $D = 16$ en toda la jerarquía (hasta 43% en RAM profunda). Para $D = 64$ la inversión de resultado evidencia el coste de un `ind[]` de gran tamaño al duplicarse R.
10. **Los valores se estabilizan en RAM profunda.** La diferencia entre $L = 1\,572\,864$ y $L = 3\,145\,728$ es inferior al 2%, confirmando que el sistema ha alcanzado la latencia de RAM pura.

A Configuración Slurm

```
#!/bin/bash
#SBATCH -n 1            # 1 proceso (sin MPI)
#SBATCH -c 64           # 64 cores logicos = nodo completo reservado
#SBATCH --mem=4G        # Memoria suficiente para los vectores grandes
#SBATCH -t 00:02:00     # Tiempo estimado de ejecucion
#SBATCH --job-name p1acg_eqNN

# -c 64 reserva el nodo completo aunque solo se use 1 core.
# Garantiza exclusividad de L1/L2 (privadas) y evita
# contaminacion de L3 (compartida) por otros procesos.
gcc acp1.c -o acp1 -O0
```

Listing 3: Cabecera de los scripts Slurm (idéntica en las tres variantes)